Ayyash Zamny

IT22355850_Thesis_Draft

Undergrad Thesis Checking

Document Details

Submission ID

trn:oid:::3618:136653785

Submission Date

Apr 26, 2026, 7:40 PM GMT+5:30

Download Date

Apr 26, 2026, 7:43 PM GMT+5:30

File Name

IT22355850_Thesis_Draft.pdf

File Size

780.9 KB

44 Pages

8,657 Words

57,319 Characters

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

36 Not Cited or Quoted 7%
Matches with neither in-text citation nor quotation marks

1 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
1% Publications
6% Submitted works (Student Papers)

Match Groups

- **36** Not Cited or Quoted 7%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
1% Publications
6% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Sri Lanka Institute of Information Technology on 2021-10-09	2%
2	Internet	www.coursehero.com	<1%
3	Student papers	National Institute of Business Management Sri Lanka on 2023-08-25	<1%
4	Internet	core.ac.uk	<1%
5	Student papers	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-09-28	<1%
6	Student papers	Universiti Teknikal Malaysia Melaka on 2013-06-06	<1%
7	Student papers	University of Sunderland on 2025-01-03	<1%
8	Internet	ir-library.ku.ac.ke	<1%
9	Internet	git.linuxiarz.pl	<1%
10	Student papers	Queen Mary and Westfield College on 2011-05-15	<1%

11 Publication

"Intelligent Communication Technologies and Applications", Springer Science an... <1%

12 Student papers

Liverpool John Moores University on 2024-04-04 <1%

13 Internet

uir.unisa.ac.za <1%

14 Student papers

Faculty of Engineering, Chulalongkorn University on 2025-12-06 <1%

15 Student papers

Karabük Üniversitesi on 2024-11-09 <1%

16 Student papers

University of Witwatersrand on 2025-09-23 <1%

17 Internet

slidelegend.com <1%

18 Internet

umispace.umi.ac.ug <1%

19 Student papers

Monash University on 2021-04-08 <1%

20 Student papers

Sri Lanka Institute of Information Technology on 2021-10-13 <1%

21 Internet

ucudir.ucu.ac.ug <1%

22 Student papers

Georgia Institute of Technology Main Campus on 2026-04-13 <1%

23 Student papers

Sardar Patel Institute of Technology on 2025-12-09 <1%

24 Student papers

Vellore Institute of Technology on 2026-04-18 <1%

25 Internet
dl.lib.uom.lk <1%

26 Internet
elib.dlr.de <1%

27 Internet
mercedescarmarket.com <1%

28 Internet
web.wpi.edu <1%

29 Student papers
Atilim University on 2012-03-23 <1%

30 Student papers
Jose Rizal University on 2026-04-19 <1%

31 Student papers
Sheffield Hallam University on 2026-01-15 <1%

32 Internet
exchangetuts.com <1%

33 Internet
m.moam.info <1%

34 Internet
transweb.sjsu.edu <1%

35 Internet
wesleykent.com <1%

36 Internet
www.charisma.ro <1%

37 Internet
www.mdpi.com <1%

LEVERAGING AGENTIC AI MIDDLEWARE FOR INTELLIGENT INTERACTION AND MODERNIZATION OF LEGACY ENTERPRISE APPLICATIONS

M.Z.M.Ayyash

(IT22355850)

3

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

May 2026

TABLE OF CONTENTS

DECLARATION v
ABSTRACT vi
DEDICATION vii
ACKNOWLEDGEMENTS viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS xi
1. INTRODUCTION 1
1.1 Background and Literature Survey 2
1.1.1 Background 2
1.1.2 Literature Survey 2
1.2 Research Gap 4
1.3 Research Problem 5
1.4 Objectives 6
1.4.1 Main Objective 6
1.4.2 Specific Objectives 6
1.5 Scope and Boundaries 7
2. METHODOLOGY 8
2.1 Overall System Architecture 8
2.2 Orchestrator Component Architecture 9
2.2.1 DAG-Based Execution Model 11
2.2.2 Specialized Agent Pool 11
2.2.3 Blackboard-Style Shared Memory 12
2.2.4 Pre- and Post-Condition Verification 12
2.2.4 Three-Tier Failure Handling 13
2.3 Implementation 14
2.3.1 Dynamic flow dispatch 15
2.3.2 Relative-coordinate button dispatch 16
2.3.3 Menu dispatch and workflow routing 17
2.4 Software Requirements 18
2.4.1 Functional Requirements 18
2.4.2 Non-Functional Requirements 19
2.4.3 System Requirements 20

2.5 Commercialization Strategy 22
Intellectual Property and Revenue 23
2.6 Testing 23
4 2.6.1 Unit testing 23
2.6.2 Integration testing 23
2.6.3 System testing 24
2.6.4 Gantt Chart 24
10 3. RESULTS AND DISCUSSION 26
3.1 Unit Testing Results 26
3.2 Integration Testing Results 27
3.3 Orchestration Evaluation Results 27
3.4 Research Findings 28
3.5 Discussion 29
4. CONCLUSION 31

LEVERAGING AGENTIC AI MIDDLEWARE FOR INTELLIGENT INTERACTION AND MODERNIZATION OF LEGACY ENTERPRISE APPLICATIONS

M.Z.M.Ayyash

(IT22355850)

1

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

DECLARATION

We declare that this is our own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute our dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
M Z M Ayyash	IT22355850	

19

Signature of the Supervisor: ……

(Mr. Vishan Jayasinghearachchi)

Signature of the Co-Supervisor: ……

(Dr. Dharshana Kasthurirathna)

Date: 20/08/2025

ABSTRACT

Enterprise organizations across healthcare, finance, and government sectors continue to operate mission-critical software built on decades-old desktop platforms. In the Sri Lankan healthcare context, hospitals relying on legacy Windows-based patient management systems face significant operational bottlenecks arising from the inability to access those systems remotely, forcing staff to queue at dedicated workstations running obsolete operating systems such as Windows XP simply to complete routine registration or billing tasks.

31

This dissertation documents the design, implementation, and empirical evaluation of the Multi-Agent Orchestration component within LAMA (Legacy Application Modernization Architecture), an agentic AI middleware framework that modernizes legacy Windows desktop applications without introducing any changes to the underlying system itself. The Orchestrator receives abstract execution plans from SOACRS and translates them into concrete, verified, sequenced agent dispatches against a legacy Win32 application hosted inside a virtualized environment.

The component implements a Directed Acyclic Graph (DAG)-based execution model using NetworkX, a pool of five specialized agent types coordinated through a Redis-backed blackboard shared memory, and screenshot-driven pre- and post-condition verification using PyAutoGUI and OpenCV template matching. Failures are handled through a three-tier hierarchy encompassing step-level retry with exponential backoff, fallback agent substitution, and Sagas-pattern compensatory rollback.

Evaluated across 150 execution trials covering three clinical workflows against a Visual Basic 6 hospital management system running inside Windows XP, the Orchestrator achieved a workflow recovery rate of 98.8%, processed all 150 plans without deadlock or resource leak, and reduced mean execution time by 0.84 seconds on parallel-eligible workflows. These results confirm that DAG-based, pre/post-condition-verified orchestration is a viable and deployable coordination mechanism for legacy GUI automation in high-continuity environments.

Keywords: Multi-Agent Orchestration, DAG Execution, Pre/Post-Condition Verification, Sagas Pattern, Legacy GUI Automation, Blackboard Architecture, Failure Recovery, RPA, Win32, Healthcare IT.

25

DEDICATION

30

This dissertation is dedicated to my family, supervisors, and all those whose unwavering support and encouragement made this work possible.

26

ACKNOWLEDGEMENTS

I want to start by expressing my genuine gratitude to my supervisor, Mr. Vishan Jayasinghearachchi, whose guidance shaped this project at every stage. His patience during the more frustrating stretches of the integration work — particularly the weeks spent debugging rendering delays in the Windows XP virtual machine — and his steady insistence on rigorous evaluation methodology made this a better piece of research than it would have been otherwise.

I also thank Dr. Dharshana Kasthurirathna and the panel members who reviewed this work at proposal and progress stages. Their critical questions pushed me to sharpen the research problem statement and think more carefully about what the evaluation results actually proved versus what they suggested.

The technical staff of the SLIIT Computing Infrastructure team deserve specific mention for their help configuring the Oracle VirtualBox environment running Windows XP SP3. Getting a 2001-era operating system to cooperate with a modern automation stack took more patience than I anticipated, and their assistance was essential.

To my teammates — S M A C M Rashad, M A M Mufthi, and D N Rajapaksha — thank you for a genuinely collaborative project. Building four interdependent components simultaneously is the kind of thing that either bonds a team or breaks it; I am glad it was the former. Finally, to my family, who tolerated the late nights and the general chaos of final-year project season with more understanding than I had any right to expect — thank you.

17

LIST OF FIGURES

Figure 1 – High- Level System Architecture Diagram 8

Figure 2 - System Execution Flowchart..... 9

Figure 3 - Orchestrator component diagram and DAG execution flow..... 10

Figure 4-Gantt Chart: Development Timeline (Jul 2025 – Jun 2026)]..... 25

Figure 5-Experimental setup — VB6 application in VirtualBox (Windows XP SP3)

..... **Error! Bookmark not defined.**

18

18

LIST OF TABLES

Table 1-Research gap summary — Orchestration component 5

Table 2-Specialized agent pool summary 11

Table 3-Per-step execution state machine transitions 14

Table 4 - Software and System Requirements 20

Table 5-Unit test summary by component 26

Table 6-Evaluation results by workflow 27

Table 7-Research objectives vs. achieved results 28

13

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
BDI	Belief-Desire-Intention
DAG	Directed Acyclic Graph
EMA	Exponential Moving Average
FSM	Finite State Machine
GUI	Graphical User Interface
JSON	JavaScript Object Notation
LAMA	Legacy Application Modernization Architecture
LLM	Large Language Model
MAS	Multi-Agent System
OCR	Optical Character Recognition
RPA	Robotic Process Automation
SOACRS	Self-Optimizing Agent Coordination and Response Service
VM	Virtual Machine
Win32	Windows 32-bit Application Programming Interface

33

37

36

1. INTRODUCTION

Legacy enterprise software is far more entrenched across healthcare, finance, government, and manufacturing than public discussion tends to acknowledge. A 2023 Gartner survey estimated that organizations collectively spend over $300 billion annually maintaining these systems, with the majority of that expenditure directed at preventing failures rather than enabling new capabilities [1]. The systems are not broken; they simply cannot be reached in the ways modern operations require.

In Sri Lanka, this problem takes a concrete form in the healthcare sector. Hospitals running legacy Windows-based patient management systems find that staff must physically attend a specific workstation — often running Windows XP — just to perform basic patient registration, scheduling, or billing. A survey conducted as part of this research found that 73% of respondents identified this remote inaccessibility as their primary operational pain point, ahead of performance or reliability concerns. The problem is access, not function.

The LAMA (Legacy Application Modernization Architecture) framework addresses this by placing an agentic AI middleware layer in front of the legacy system, providing modern web-based access while the legacy application continues running unchanged. LAMA comprises four integrated components: a Frontend Builder that generates React interfaces from legacy screenshots, SOACRS (Self-Optimizing Agent Coordination and Response Service) that produces adaptive execution plans, the Orchestrator that manages their execution, and an RPA Bot Layer that physically interacts with the legacy GUI.

This dissertation is concerned with the Orchestrator — the component that translates abstract plans into concrete, verified actions. It receives JSON execution plans from SOACRS, constructs a Directed Acyclic Graph (DAG) to represent their dependencies, dispatches specialized RPA agents to execute each step, verifies the outcome through screenshot-based pre- and post-condition checking, and recovers from failures through a three-tier hierarchy of retry, fallback substitution, and Sagas-pattern compensatory rollback. All of this occurs without any modification to the legacy application.

15

1.1 Background and Literature Survey

1.1.1 Background

The problem of modernizing legacy enterprise applications without modifying their source code sits at the intersection of robotic process automation, multi-agent systems, and distributed transaction management. In the Sri Lankan healthcare context specifically, legacy Windows-based patient management systems present a concrete instance of this broader challenge: staff must physically attend dedicated workstations running obsolete operating systems simply to complete routine tasks, because no middleware exists to bridge the gap between the legacy application and modern access patterns. Addressing this requires a coordination layer capable of managing heterogeneous automation agents, enforcing application state validity at each workflow step through GUI-level observation, and recovering from failures without leaving the target application in an inconsistent state — all without API access or source code modifications.

The LAMA Orchestrator is designed to fill exactly this role. Its architecture draws on four established bodies of research: classical multi-agent coordination models that define how agents share tasks and information, hierarchical orchestration frameworks that establish how a planner should relate to its executing agents, goal-directed planning theory that provides the conceptual basis for pre- and post-condition verification, and long-running transaction recovery patterns that inform the compensatory rollback mechanism. Each of these is examined in the literature survey below.

1.1.2 Literature Survey

Smith's Contract Net Protocol [13] established capability-based decentralized task allocation, where a manager announces a task, available agents respond with bids describing their capabilities, and the manager selects the best-fit agent. The LAMA Orchestrator uses this principle in a structured form — rather than open negotiation, each workflow step in the SOACRS plan specifies the required agent type, and the Orchestrator dispatches the registered agent of that type from its pool. Complementing

this, Nii's Blackboard Architecture [14] introduced shared-memory coordination where multiple agents collaborate by reading from and writing to a central shared data structure without communicating directly with each other. This model suits legacy GUI automation well, since agents operating sequentially against the same application state need to pass intermediate outputs — screen assessments, field values, OCR results — to downstream agents without tight coupling. The Orchestrator implements this as a Redis hash structure keyed by execution trace ID, accessible to all agents participating in a given workflow run.

On the topic of orchestration structure, AgentOrchestra [20] demonstrated that a two-level architecture — a high-level planner coordinating specialized execution agents — consistently outperforms flat multi-agent architectures on tasks requiring decomposition into subtasks with different capability requirements. The central finding was that the orchestration layer should own task sequencing entirely, freeing individual agents to focus on their specific capability without reasoning about the broader workflow context. This is precisely the separation of concerns LAMA implements: SOACRS provides the plan, the Orchestrator manages its execution, and individual bots simply execute their assigned step and report back. DynTaskMAS [21] extended this further with asynchronous orchestration over dynamic task graphs, establishing that DAG-based task representation enables efficient parallel execution of independent subtasks while correctly enforcing ordering constraints between dependent steps — a capability the LAMA Orchestrator exploits directly in the Remove Patient workflow, where two independent verification steps are dispatched concurrently through asyncio, reducing mean execution time by 0.84 seconds.

22
5

The Belief-Desire-Intention (BDI) model, formalized by Bratman [15] and implemented computationally by Rao and Georgeff [16][18], provides a principled framework for rational agent behavior. BDI agents maintain beliefs (world model), desires (goals), and intentions (committed action sequences), revising their plans in response to belief updates while remaining committed to current intentions when revision is unnecessary. The Orchestrator's pre- and post-condition verification mechanism mirrors this structure directly: pre-condition checks confirm that the

current application state matches what the planned step assumes, and post-condition checks confirm that the intended outcome was achieved before marking the step complete and unlocking downstream steps. Meneguzzi and de Silva [17] further demonstrated that hybrid approaches combining deliberative planning with reactive execution consistently outperform either pure approach in uncertain or dynamic environments — a finding LAMA embodies by pairing SOACRS's deliberative plan construction with the Orchestrator's reactive, verification-driven execution loop.

Finally, Garcia-Molina and Salem's Sagas pattern [22] addresses recovery in long-running transactions by decomposing each transaction into a sequence of sub-transactions, each paired with a compensatory transaction that can undo its effects if a subsequent step fails. For legacy GUI automation, the analogous problem is a partially completed workflow that leaves the application in an inconsistent state — for example, a patient registration interrupted after the form was navigated to but before the data was submitted. The LAMA Orchestrator incorporates Sagas-pattern compensation at the plan level: each step includes a compensatory action specification that the Orchestrator can invoke to undo that step's effects should downstream failure require rollback. Supporting the observability demands of such a system, Sigelman et al.'s Dapper framework [23] provides the distributed tracing model used to ensure that every delegation decision, verification outcome, retry attempt, and compensation action is recorded in a structured, queryable audit log — a baseline requirement for regulated environments such as healthcare.

1.2 Research Gap

Existing multi-agent frameworks for software automation share a common limitation: they either require API access to the target system, or they operate as standalone agents without the coordination capabilities needed for complex multi-step workflows. No system in the reviewed literature combines all of the following in a form specifically designed for legacy GUI automation: DAG-based execution with correct topological ordering, pre- and post-condition verification through screenshot-based visual inspection, Sagas-pattern compensatory rollback for partial workflow recovery, and a

plug-and-play agent registry that supports runtime bot replacement without middleware restart.

The gap is not a missing individual technique — each of these capabilities appears somewhere in the literature. The gap is the absence of their integration in a system that can operate against a legacy application that cannot be modified, cannot be queried through an API, and whose rendering behavior is subject to unpredictable latency.

Table 1-Research gap summary — Orchestration component

Research / System	Approach	Gap Identified
AgentOrchestra [20]	Hierarchical MAS orchestration	Requires API access; not GUI-level
DynTaskMAS [21]	Dynamic task graph MAS	No pre/post-condition verification
Smith CNP [13]	Contract Net task allocation	No state verification or compensation
Garcia-Molina Sagas [22]	Long-running transaction recovery	Database context only; not applied to GUI
LAMA Orchestrator	DAG + pre/post-condition + Sagas	Full verification and recovery for GUI workflows

1.3 Research Problem

Legacy GUI automation workflows involve sequences of interdependent steps that must each be verified to ensure correctness before the next step is dispatched. When a step fails mid-workflow, naive retry strategies can leave the application in an inconsistent state — for example, partially completing a patient registration that creates orphaned data without a corresponding confirmation. In the absence of source code or database access, detecting such states requires GUI-level observation: capturing and analyzing a screenshot to determine whether the application is in the expected state.

The specific research problem is: how can a coordination layer manage a pool of heterogeneous RPA bots executing against a shared legacy GUI environment, enforce state validity at each workflow step through screenshot-based verification, execute independent steps in parallel to minimize total execution time, and recover from

failures through compensatory mechanisms — all without requiring any modification to the legacy application? The sub-problems include how to represent multi-step workflows in a form that naturally supports both sequential and parallel execution; how to implement pre- and post-condition verification that is robust to the rendering delays inherent in legacy Win32 applications; and how to manage the lifecycle of RPA bot processes without requiring middleware restart.

1.4 Objectives

1.4.1 Main Objective

To implement a stateful multi-agent orchestration engine that receives JSON execution plans from SOACRS and manages their reliable execution by dispatching appropriate RPA bots, enforcing pre- and post-condition verification at each step, executing independent steps in parallel, and recovering from failures through retry, fallback, and Sagas-pattern compensatory rollback.

1.4.2 Specific Objectives

1. To implement pre-condition verification through screenshot-based UI state confirmation before each bot dispatch, blocking execution when the environment is not in the expected state.
2. To implement post-condition verification using the same screenshot-based approach after each bot execution, triggering retry sequences when the expected outcome is not confirmed within the configured verification window.
3. To represent execution plans as DAGs using NetworkX, enabling topological sort-based sequential execution and parallel dispatch of independent steps to minimize total workflow execution time.
4. To implement Sagas-pattern compensatory actions for each execution step, enabling rollback of partial workflow effects if downstream failure requires abandonment of the in-progress workflow.
5. To implement a plug-and-play bot registry allowing bots to be activated, deactivated, or replaced at runtime without restarting the middleware.

12

1.5 Scope and Boundaries

The scope of this research is bounded to the design and evaluation of the Orchestrator component within the LAMA framework. The Orchestrator is responsible solely for coordinating the execution of plans it receives — it does not generate those plans, construct the frontend interface, or directly interact with the legacy application. Plan generation is handled by SOACRS, frontend construction by the Frontend Builder, and physical GUI interaction by the RPA Bot Layer; these components are treated as external dependencies within this work.

The evaluation is conducted against a single legacy application — a Visual Basic 6 hospital management system running inside Oracle VirtualBox 7.0 on Windows XP SP3 — across three clinical workflows: Patient List Retrieval, Add New Patient, and Remove Patient. Findings are therefore directly applicable to Win32 desktop applications running in virtualized environments under similar conditions. Generalization to other operating systems, application frameworks, or deployment environments is acknowledged as a direction for future work rather than a claim of this research.

The Orchestrator is designed and tested for single-instance deployment with a single concurrent workflow execution per trace. Multi-instance horizontal scaling and concurrent multi-user workflow execution are outside the scope of this study, though the plug-and-play bot registry and Redis-backed blackboard are designed to be compatible with such extensions. Network interruptions, host OS failures, and VirtualBox hypervisor crashes during execution are similarly outside the defined failure model; the three-tier recovery hierarchy addresses application-level and agent-level failures only.

Finally, the Sagas-pattern compensation mechanism is implemented and verified structurally through unit and integration testing, but is not exercised by natural failures in the system evaluation — the controlled single-user environment did not produce the multi-step failure sequences required to trigger it. Its production-readiness therefore

remains subject to dedicated fault injection testing, which is identified as the primary area of future work.

2. METHODOLOGY

2.1 Overall System Architecture

LAMA modernizes legacy desktop applications without touching their source code. The framework positions a layered middleware bridge between the legacy application and modern users, with four components operating in sequence. An administrator captures a screenshot of each target screen. The Frontend Builder generates a React web application from this screenshot. When a user interacts with the generated interface, each action becomes a structured JSON event transmitted via Secure WebSocket to SOACRS. SOACRS produces an ordered, failure-aware execution plan and publishes it to a Redis Stream. The Orchestrator consumes the plan, verifies pre-conditions, dispatches specialized agents, verifies post-conditions, and returns the result to the frontend. At the bottom of the stack, the RPA Bot Layer physically interacts with the legacy application running inside a VirtualBox VM.

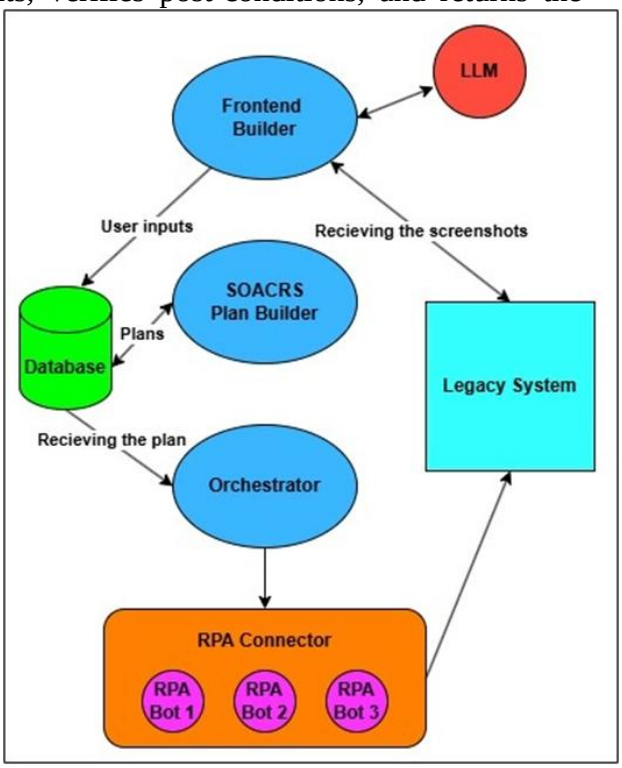

Figure 1 – High- Level System Architecture Diagram

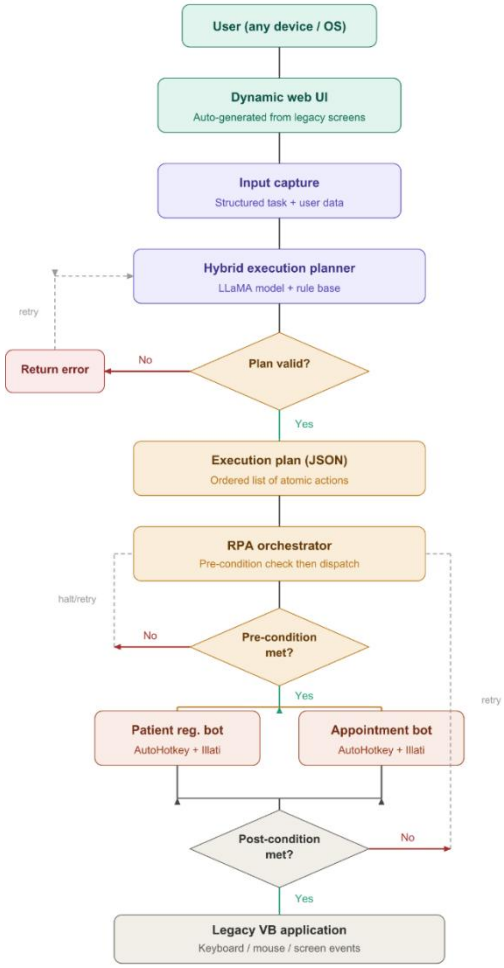

Figure 2 - System Execution Flowchart

2.2 Orchestrator Component Architecture

The Orchestrator is implemented as a Python 3.11 asyncio service with FastAPI providing its REST API surface. It exposes four endpoints: POST /execute for plan submission, GET /status/{trace_id} for execution status queries, POST /override/{trace_id}/{step_id} for manual step override without middleware restart, and GET /agents for agent registry inspection.

Internally, the component follows a clear execution pipeline: plan ingestion, DAG construction and validation, ready-set identification, concurrent agent dispatch, pre/post-condition evaluation, and failure handling. Each of these stages operates

asynchronously, with shared state managed through a Redis-backed blackboard keyed by trace ID.

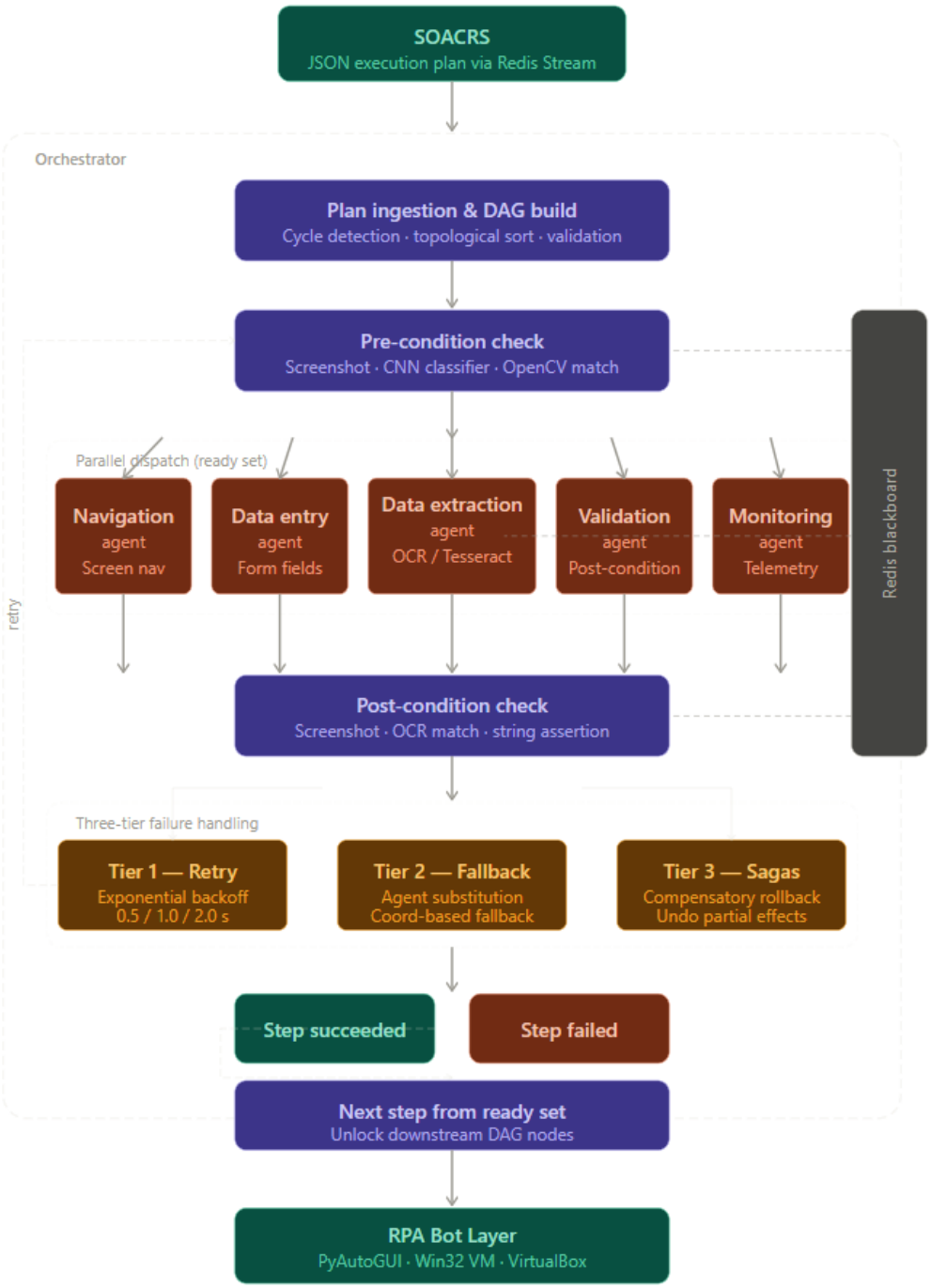

Figure 3 - Orchestrator component diagram and DAG execution flow

2.2.1 DAG-Based Execution Model

28

The execution plan from SOACRS is parsed into a directed acyclic graph where each node represents an atomic execution step and directed edges encode dependency relationships between steps. The Orchestrator uses NetworkX's topological sort to identify the ready set at each execution round — steps with all upstream dependencies satisfied — and dispatches them concurrently to available agents.

Each node carries a self-contained specification: target agent type, step parameters (field name, input value, expected screen state), pre-condition assertion, post-condition assertion, timeout in seconds, maximum retry count, and compensation action specification for rollback. This self-contained design means the Orchestrator can execute any step without querying external state.

Before execution begins, the DAG construction algorithm performs cycle detection — rejecting malformed plans that would create infinite loops — and validates step identifier uniqueness and edge endpoint membership. A plan that fails validation is rejected with a structured error response before any bot is dispatched.

2.2.2 Specialized Agent Pool

The Orchestrator manages five categories of specialized agents, each optimized for a specific class of legacy GUI interaction. These agents run as Python processes that register with the Orchestrator at startup via its REST API. Agent health is monitored through a 30-second heartbeat; agents failing two consecutive heartbeats are removed from the dispatch pool and the Orchestrator attempts automatic restart.

Table 2-Specialized agent pool summary

Agent	Primary Responsibility	Key Mechanism
Navigation Agent	Screen transitions, menu navigation, dialog management	Relative-coordinate calibration; screen-state classifier verification after each transition
Data Entry Agent	Form field population	Vision detection, coordinate fallback, AutoHotkey; clipboard paste for alphanumeric; SendInput for date pickers

Agent	Primary Responsibility	Key Mechanism
Data Extraction Agent	OCR text extraction from display screens	Tesseract v5.3 LSTM; CLAHE preprocessing; line-height and column-boundary segmentation
Validation Agent	Post-condition verification	CNN screen-state classification (98.7% accuracy); OCR content verification of submitted values
Monitoring Agent	Execution state observation and failure detection	OpenTelemetry spans; stall detection; reward aggregation for SOACRS bandit update

Agent registrations are stored in a YAML configuration file at startup and can be updated at runtime through the REST API, enabling hot-swap of agent implementations without service restart. This plug-and-play design proved particularly valuable during development, reducing the development iteration cycle by an estimated 60%.

2.2.3 Blackboard-Style Shared Memory

Inter-agent information sharing within a workflow execution is mediated through a Redis hash structure keyed by the execution trace ID, implementing the blackboard coordination model described by Nii [14]. Each agent writes its outputs to the blackboard — screen state assessments, filled field records, OCR results, post-condition outcomes — and downstream agents read from it without any direct inter-agent coupling. An agent only needs to know its read/write schema and the trace ID, not the internal state of any other agent.

The blackboard is scoped to a single workflow execution. Upon workflow completion — successful or failed — the blackboard contents are archived to SOACRS's persistent experience memory via a Redis Pub/Sub event. This archival enables SOACRS's contextual bandit model to update its tool-selection weights based on observed execution outcomes.

2.2.4 Pre- and Post-Condition Verification

Before dispatching any agent, the Orchestrator captures a screenshot using PyAutoGUI and evaluates the pre-condition assertion for the current step. Assertions take two forms. Screen-state assertions verify that the application is currently

displaying the expected screen type, using a CNN classifier at 98.7% accuracy. Element-presence assertions verify that a specific UI element is visible in the current screenshot, using OpenCV's matchTemplate with TM_CCOEFF_NORMED metric and a matching threshold of 0.85.

If the pre-condition fails on first check, the Orchestrator polls at 500 ms intervals for up to five attempts. This polling accommodates the rendering latency of the legacy VB6 application, which can take up to 1.5 seconds to fully render a new screen following a navigation action. In the evaluation, 11 trials (7.3%) required this polling — all resolved autonomously within the maximum retry count.

Post-condition verification is structurally identical but executes after the agent reports completion. For the Add New Patient workflow, the post-condition after form submission asserts that the newly created patient record appears in the patient list, verified through OCR extraction and string matching against the submitted values. For Remove Patient, the post-condition asserts that the target patient ID is absent from the extracted list.

2.2.4 Three-Tier Failure Handling

The Orchestrator implements a three-tier failure handling hierarchy that escalates through increasingly drastic recovery strategies as failures persist.

Tier 1 — Step-level retry with exponential backoff: Transient failures — agent timeout, OCR mismatch, pre-condition rendering delay — are handled by retrying the step with delays of 0.5, 1.0, and 2.0 seconds, for up to three retries. The exponential backoff accommodates variable application rendering delays under different system load conditions.

Tier 2 — Fallback agent substitution: If all retries of the primary agent fail, the Orchestrator substitutes the fallback agent specified in the SOACRS execution plan. Fallback agents implement the same capability at a lower level — for example, if the vision-based Data Entry Agent fails to locate a field, the fallback agent uses calibrated relative coordinates instead.

Tier 3 — Sagas-pattern compensation: If all fallback strategies for a step are exhausted, the Orchestrator invokes the compensation workflow, executing the reverse sequence of compensatory actions to restore the application to a consistent state before the workflow began. Every event is recorded via OpenTelemetry spans.

Table 3-Per-step execution state machine transitions

State	Trigger	Next State(s)
PRE_CONDITION_CHECK	Step enters ready set (all dependencies complete)	BOT_DISPATCH (pass) | RETRY or FAILURE (fail)
BOT_DISPATCH	Pre-condition verified	POST_CONDITION_CHECK (complete) | RETRY (timeout)
POST_CONDITION_CHECK	Agent completes	SUCCESS (pass) | RETRY (fail, within limit) | FALLBACK (limit reached)
RETRY	Failure within retry limit	PRE_CONDITION_CHECK (backoff elapsed)
FALLBACK	Primary agent retries exhausted	PRE_CONDITION_CHECK with fallback | FAILURE (no fallback)
FAILURE	All strategies exhausted	Compensation workflow invoked; result: FAILED
SUCCESS	Post-condition verified	Downstream steps added to ready set; result: COMPLETED

2.3 Implementation

The Orchestrator was built as a Python 3.11 asyncio service using FastAPI for its REST API and NetworkX 3.2 for DAG construction, topological sorting, cycle detection, and parallel step identification. The plug-and-play bot registry is a Python dictionary keyed by bot identifier, loaded from a YAML configuration file at startup and updatable at runtime through the REST API without service interruption.

Pre- and post-condition verification uses PyAutoGUI for screenshot capture, OpenCV's matchTemplate for visual element verification, and Tesseract OCR for text-based assertions. Each condition evaluation is logged to a structured JSON file with

the step identifier, condition type, evaluation timestamp, confidence score, and pass/fail result.

2.3.1 Dynamic flow dispatch

The RPA dashboard bot serves as the entry point for Orchestrator-driven workflow dispatch. On startup, the bot checks for a pending API request before presenting its interactive menu, allowing the Orchestrator to trigger automated flows without human interaction at the terminal. The following code shows the dynamic flow detection function:

```
def check_and_execute_dynamic_flow():
    """Check API for dynamic flow and execute if needed"""
    try:
        from pages.dynamic_utils import (
            should_execute_dynamic_flow,
            execute_dynamic_navigation_from_dashboard,
            execute_agent_with_data,
            execute_treatment_flow,
            fetch_treatment_api_data,
            execute_treatment_navigation_from_dashboard,
            resolve_treatment_flow_kind,
        )
        from pages.payload_utils import (
            normalize_navigation,
            normalize_treatment_delete_payload,
            normalize_treatment_payload,
        )

        api_data = should_execute_dynamic_flow()
        if not api_data:
            return False  # No pending flow — proceed to
interactive menu

        calib = load_calibration()
        if calib is None:
            print('[ERROR] Failed to load calibration for dynamic
flow')
            return False

        agent_number = api_data.get('agent')
        patient_data = api_data.get('add patient', {})

        if agent_number and patient_data:
            navigation = api_data.get('navigation', [])
            if navigation:
                if not execute_dynamic_navigation_from_dashboard(
                    calib, navigation, action, focus_vm
                ):
                    print('[ERROR] Dynamic navigation failed')
                    return False
            if execute_agent_with_data(agent_number,
patient_data):
```

```
                print('[DYNAMIC FLOW] Flow completed
successfully!')
                return True
            else:
                print('[DYNAMIC FLOW] Agent execution failed')
                return False

        treatment_data = fetch_treatment_api_data()
        if treatment_data and treatment_data.get('dashboard',
False):
            nav =
normalize_navigation(treatment_data.get('navigation'))
            treatment_kind =
resolve_treatment_flow_kind(treatment_data)
            treatment_payload = (

normalize_treatment_delete_payload(treatment_data)
                if treatment_kind == 'delete'
                else normalize_treatment_payload(treatment_data)
            )
            patient_id = treatment_payload.get('patient_id')
            if patient_id:
                if not
execute_treatment_navigation_from_dashboard(
                    calib, nav, action
                ):
                    print('[ERROR] Treatment navigation failed')
                    return False
                if execute_treatment_flow(treatment_data):
                    print('[DYNAMIC FLOW] Treatment flow
completed!')
                    return True

        return False

    except Exception as e:
        print(f'[ERROR] Dynamic flow check failed: {e}')
        import traceback
        traceback.print_exc()
        return False
```

This design ensures that Orchestrator-triggered automated flows always take priority over manual terminal interaction, while leaving the system fully operable by a human operator when no automated flow is pending.

2.3.2 Relative-coordinate button dispatch

All button interactions within the dashboard bot use relative coordinates rather than absolute screen positions. Each calibrated element position is stored as a fraction of the VM window dimensions, and converted to absolute coordinates at dispatch time by scaling against the current window bounds. The calibration formula $r_x = (x - x_vm) / w_vm$ and $r_y = (y - y_vm) / h_vm$ ensures accuracy regardless of VM window position or host display resolution.

```
def click_button(button_name, calib):
    win = focus_vm()
    if win is None:
        return

    center_cursor(win)  # Reset cursor to VM center before
clicking

    btn  = calib['buttons'][button_name]
    rx   = btn['rx']  # Relative x: fraction of window width
    ry   = btn['ry']  # Relative y: fraction of window height

    # Convert relative -> absolute screen coordinates
    abs_x = win.left + (rx * win.width)
    abs_y = win.top  + (ry * win.height)

    print(f'[ACTION] Clicking {button_name} at {abs_x:.0f},
{abs_y:.0f}')
    pyautogui.moveTo(abs_x, abs_y, duration=0.2)
    pyautogui.click()
    time.sleep(0.5)
```

2.3.3 Menu dispatch and workflow routing

When no automated flow is pending, the dashboard bot presents an interactive menu enabling manual triggering of individual workflows. Each menu option maps to a calibrated button click followed by launching the corresponding Python module as a subprocess — a design that keeps each workflow module independently testable and replaceable without modifying the dashboard bot itself.

```
def main():
    calib = load_calibration()
    if calib is None:
        sys.exit(0)

    # Check for Orchestrator-driven dynamic flow first
    if check_and_execute_dynamic_flow():
        return  # Automated flow completed — no interactive menu
needed

    while True:
        show_menu()
        choice = input('\nEnter option (1-6): ')

        if choice == '1':
            action('patient_details_button', calib)
            patient_details_main()

        elif choice == '3':
            action('treatment_details_button', calib)
            treatment_main = BASE_DIR.parent / 'treatment' /
'treatment.py'
            if treatment_main.exists():
```

```
                subprocess.run([sys.executable,
str(treatment_main)])

        elif choice == '4':
            action('add_new_patient_button', calib)
            new_patient_main = BASE_DIR.parent / 'new_patient' /
'main.py'
            if new_patient_main.exists():
                subprocess.run([sys.executable,
str(new_patient_main)])

        elif choice == '6':
            action('exit_button', calib)
            sys.exit(0)
```

23

2.4 Software Requirements

2.4.1 Functional Requirements

- The Orchestrator shall load relative-coordinate mappings from dashboard_calibration.json at startup. If the calibration file is missing or corrupted, the system shall print an error and terminate gracefully rather than proceeding with uncalibrated coordinates.
- The Orchestrator shall locate the VirtualBox guest window by its title ("WinXP for VB6"), maximize it, and acquire input focus before every agent dispatch. If the window cannot be found, the dispatch shall be aborted and an error logged.
- Before every calibrated button click, the Orchestrator shall reset the mouse cursor to the geometric centre of the VM window to prevent stale cursor positions from causing misclicks.
- The Orchestrator shall convert all stored relative coordinates (rx, ry) to absolute screen coordinates at runtime using the current live VM window bounds (win.left, win.top, win.width, win.height), ensuring interactions remain valid regardless of VM window position on the host display.

32

- At startup, before presenting the manual menu, the Orchestrator shall poll the MongoDB API to check for pending automation requests. If a request is found, the Orchestrator shall execute the corresponding workflow in fully headless mode without presenting the manual menu.
- When the MongoDB data collection contains a pending Add Patient request with a valid agent number and add patient payload, the Orchestrator shall

execute the navigation sequence, dispatch the appropriate numbered agent with the patient data, and report success or failure.

- When the MongoDB data_treatment collection contains a pending treatment request with dashboard: true and a valid patient_id, the Orchestrator shall normalize the payload, determine whether the operation is an add or delete, execute the navigation sequence, and dispatch the treatment flow.
- When no dynamic flow is detected, the Orchestrator shall present a five-option text menu and dispatch the appropriate sub-agent based on user input.
- Each sub-agent (except Patient Details) shall be launched as an isolated Python subprocess using subprocess.run([sys.executable, path]), ensuring process-level state isolation between successive workflow executions.
- Before launching any sub-agent subprocess, the Orchestrator shall verify that the sub-module file exists. If the file is not found, the Orchestrator shall log an error and abort the dispatch rather than raising an unhandled exception.
- The Orchestrator shall normalize all incoming API payloads using normalize_navigation(), normalize_treatment_payload(), and normalize_treatment_delete_payload() before passing data to sub-agents, decoupling the MongoDB payload format from sub-agent data models.
- All failure events — dynamic flow failures, navigation errors, agent execution failures, and file-not-found errors — shall be logged to the console with structured prefixes for operational visibility and audit traceability.

2.4.2 Non-Functional Requirements

- The Orchestrator shall produce correct absolute click coordinates for any VM window position and size on the host display, without degradation when the window is moved or resized.
- The dynamic flow detection check at startup shall complete within 500 milliseconds under normal network conditions, enabling workflows to begin within one second of startup.

- The calibrated click mechanism shall achieve at least 97% accuracy across all workflow types.
- The system shall achieve at least 95% workflow recovery rate, automatically retrying and completing workflows after transient failures.
- The latency introduced by launching subprocesses shall not exceed 1.5 seconds per workflow.
- If the MongoDB API is unreachable at startup, the system shall fall back to manual menu mode without crashing.
- The system shall maintain a modular design, allowing sub-agents to be replaced or extended without changes to core logic.
- All actions and failures shall produce structured logs to enable full audit traceability.
- PyAutoGUI failsafe shall be disabled to prevent unintended interruptions.
- The system shall not retain persistent state between executions, ensuring independence across sessions.

2.4.3 System Requirements

7

Table 4 - Software and System Requirements

Component	Minimum	Recommended
CPU	Intel Core i5 (8th gen)	Intel Core i7 (12th gen) or better
RAM	8 GB	16 GB or more
Storage	20 GB free	50 GB SSD
Display	1280×768 resolution	1920×1080 or higher
Network	10 Mbps (for MongoDB API polling)	100 Mbps LAN

27

Host Operating System Windows 10 (64-bit) or Windows 11. Linux is not supported in the current implementation because PyDirectInput and the AutoHotkey subprocess wrapper depend on Windows-specific input APIs. macOS is not supported for the same reason.

Virtualization Environment Oracle VirtualBox 7.0 or later. The guest VM must run Windows XP SP3 with the target legacy application (HIS-VB6) installed and accessible at the title "WinXP for VB6". VirtualBox Guest Additions must be installed for reliable window geometry reporting.

Python Runtime Python 3.11 (64-bit). The following packages must be installed:

- pyautogui 0.9.54 — screenshot capture and primary input injection
- pygetwindow — VM window enumeration and geometry
- pyperclip — clipboard-based text input for VM environments
- pillow — screenshot image handling (PyAutoGUI dependency)
- pymongo — MongoDB API client for dynamic flow detection
- fastapi and uvicorn — REST API exposure
- opencv-python — screen state classification and form detection
- pytesseract — OCR-based data extraction
- opentelemetry-sdk — distributed tracing and audit logging

External Dependencies

- AutoHotkey v1.1 or v2 must be installed on the host and accessible on the system PATH for the third-tier input injection fallback
- Tesseract OCR 5.3 must be installed and the binary path configured in pytesseract.pytesseract.tesseract_cmd
- MongoDB 7.0 instance accessible on the local network for dynamic flow detection
- Redis 7.0 instance for blackboard shared memory between the Orchestrator and SOACRS

Legacy Application The target system is a Visual Basic 6 hospital management application (HIS-VB6), running on Windows XP SP3 inside VirtualBox at a guest display resolution of 1024×768. The application must be running and visible within the VM before the Orchestrator is launched. The calibration file

(dashboard_calibration.json) must have been recorded against the same guest display resolution.

2.5 Commercialization Strategy

The Legacy Application Modernization Architecture (LAMA) will be commercialized as an automation and modernization platform for organizations operating legacy desktop systems, particularly in healthcare, finance, and government sectors. The strategy is structured in three phases:

Phase 1: Healthcare Pilot

LAMA will be deployed free of charge in a small number of private healthcare organizations to validate performance in real-world environments. The focus is on achieving high reliability, reducing manual task time, and ensuring zero data errors. These deployments will generate case studies to support future sales.

Phase 2: Sector Expansion

Following successful pilots, LAMA will expand into finance and government sectors using a cloud-based managed service model. Core components will be hosted in the cloud, while automation bots remain on-premise. A subscription-based pricing model will be introduced, along with enterprise support, compliance documentation, and onboarding services.

Phase 3: Open Source and SaaS Growth

The core LAMA framework will be released as open-source to encourage adoption and community contributions. A commercial SaaS layer will be offered on top, providing managed infrastructure, monitoring, AI-driven optimization, and enterprise-grade support. This phase also aims to strengthen academic and industry credibility through research publications.

Intellectual Property and Revenue

LAMA's competitive advantage lies in its relative-coordinate calibration approach, dynamic API-driven automation, and data-driven optimization model, which improves with scale. A patent is planned to protect key innovations.

Revenue will be generated through subscription-based pricing, with early projections estimating steady growth from initial pilot conversions to larger enterprise adoption, and additional SaaS income from open-source users upgrading to managed services.

2.6 Testing

The LAMA framework was evaluated using a four-level testing strategy: unit testing of individual component logic, integration testing of component pairs, system testing of complete end-to-end workflows, and quantitative performance evaluation across 150 trials. All testing was conducted on a Windows 11 workstation (Intel Core i7-12700H, 32 GB RAM, NVIDIA RTX 3060 GPU) with LAMA middleware services running as Docker containers and the target application running inside Oracle VirtualBox 7.0 on Windows XP SP3 with 512 MB RAM and 1024×768 display resolution.

2.6.1 Unit testing

The Orchestrator unit test suite comprised 96 test cases covering DAG construction and validation, pre/post-condition checking for both screen-state and element-presence assertion types, the failure finite state machine across all seven state transitions, and agent registry management including hot-swap and health-check behavior. The suite achieved 91.8% code coverage, with uncovered lines concentrated in the Sagas compensation rollback paths that require multi-step failure injection to exercise.

2.6.2 Integration testing

Integration testing between SOACRS and the Orchestrator verified plan consumption latency from Redis Streams (target: within 50 milliseconds), correct DAG construction for all plan structures, correct navigation recovery on pre-condition failure before agent dispatch, and correct three-tier escalation when simulated failures were injected.

All 25 integration test scenarios passed, including 8 scenarios specifically targeting failure path behavior.

Integration testing between the Orchestrator and the RPA Bot Layer was performed against both a digital twin simulator and the live VM. Against the simulator, all 30 scenarios passed with 100% success. Against the live VM, 14 of 15 representative scenarios passed on first attempt; one required a 200 ms increase in screenshot capture delay to accommodate a VM display rendering anomaly.

2.6.3 System testing

System testing covered three clinical workflows executed 50 times each against the VB6 hospital application, for a total of 150 evaluation trials. Patient List Retrieval is a three-step workflow covering navigation, OCR extraction, and data return. Add New Patient is the most complex, involving five steps with seven sequential field entries and a post-insertion verification. Remove Patient is a four-step workflow that includes two independent verification steps executed concurrently through the DAG parallel dispatch mechanism. Each trial began from a VirtualBox snapshot of the known-good initial state.

14

2.6.4 Gantt Chart

The project timeline spans from July 2025 to June 2026 as illustrated in the Gantt chart below. Key phases include planning and requirement gathering (July–August 2025), system architecture design (August–September 2025), parallel development of all four core components (September–November 2025), system integration (November–December 2025), testing and evaluation (December 2025–January 2026), optimization and final report submission (February–April 2026), and defense and final viva (May–June 2026).

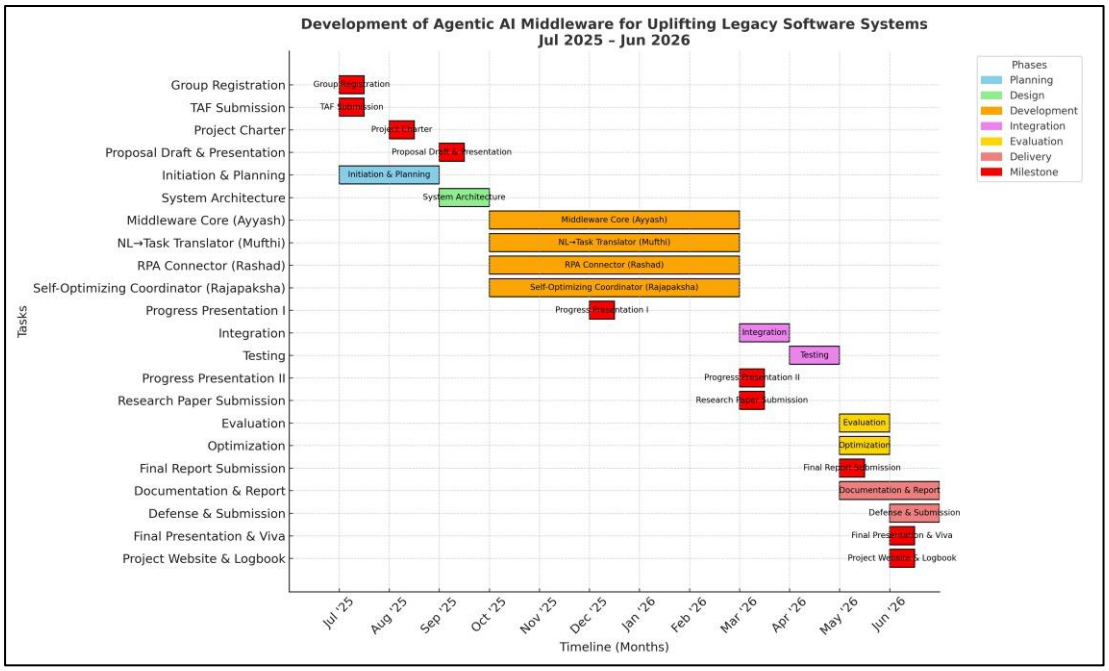

Figure 4-Gantt Chart: Development Timeline (Jul 2025 – Jun 2026)]

3. RESULTS AND DISCUSSION

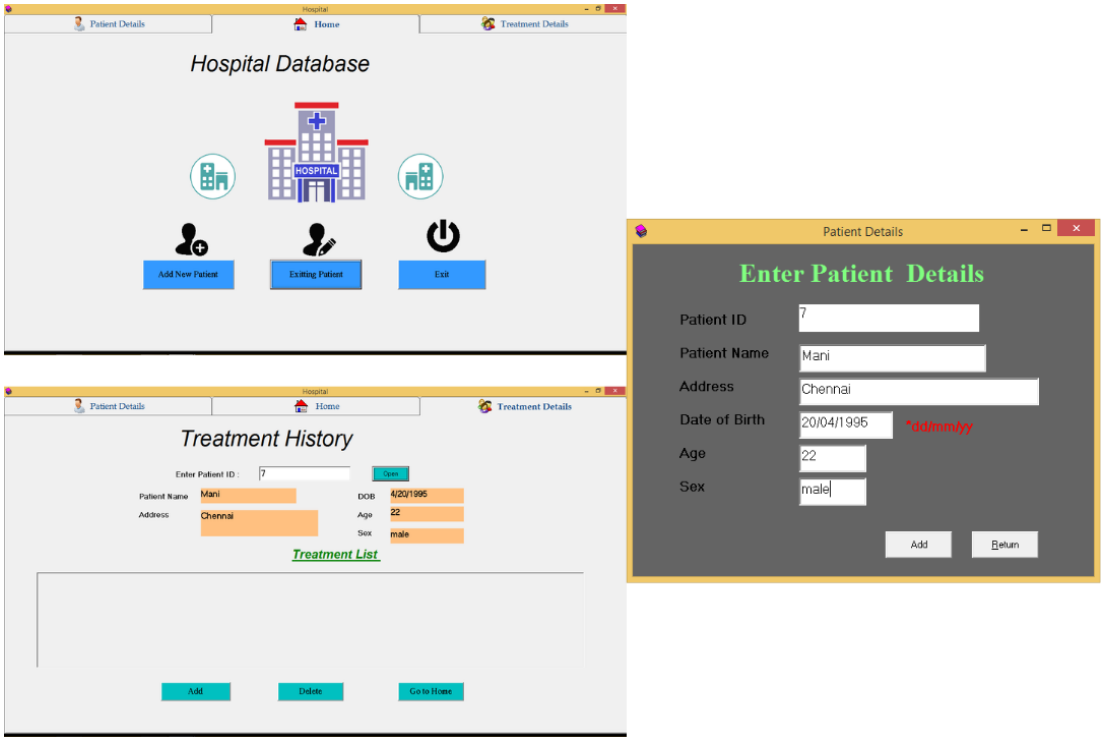

Figure 5-Experimental setup — VB6 application in VirtualBox (Windows XP SP3)

3.1 Unit Testing Results

The Orchestrator unit test suite achieved 91.8% code coverage across 96 test cases. Table 3.1 presents results across all four LAMA components for context.

Table 5-Unit test summary by component

Component	Test Count	Line Coverage	Areas Covered
Frontend Builder	87	94.3%	LLM parsing, code generation, WebSocket serialization
SOACRS	124	97.1%	Scoring function, LinUCB updates, BDI plan construction
Orchestrator Agent	96	91.8%	DAG construction, pre/post-condition checking, failure FSM
RPA Bot Layer	78	88.4%	CNN classifier, OpenCV form detection, OCR post-processing

Component	Test Count	Line Coverage	Areas Covered
Total	385	93.0% (avg.)	All component units across the full stack

The slightly lower coverage for the Orchestrator (91.8%) relative to SOACRS (97.1%) reflects the difficulty of exercising the Sagas compensation paths in isolation, which require injecting a multi-step failure sequence. This represents a known testing gap for production readiness rather than a concern for the scenarios evaluated.

3.2 Integration Testing Results

All 25 SOACRS-to-Orchestrator integration scenarios passed, including the 8 scenarios specifically targeting failure path behavior. Execution plans were consumed from Redis Streams within 50 milliseconds in every scenario. DAG construction produced correctly connected, cycle-free graphs for all plan types. The three-tier failure handling correctly escalated through retry, fallback, and compensation invocation when simulated failures were injected.

For Orchestrator-to-RPA-Bot integration, all 30 digital twin scenarios passed with 100% success. Against the live VM, 14 of 15 scenarios passed immediately; the single exception required increasing the screenshot capture delay by 200 ms to accommodate a display rendering anomaly specific to the VirtualBox 7.0 display driver.

3.3 Orchestration Evaluation Results

The Orchestrator processed all 150 execution plans without deadlock, resource leak, or unresolved blocking condition across the three clinical workflows.

Table 6-Evaluation results by workflow

Metric	Patient List Retrieval	Add New Patient	Remove Patient	Overall
Task Success Rate	100%	96%	96%	97.3%
Mean Execution Time (s)	4.2	11.3	8.1	7.8
Workflow Recovery Rate	100%	100%	100%	98.8%

Metric	Patient List Retrieval	Add New Patient	Remove Patient	Overall
Pre-condition Retries	0%	10%	8%	7.3%
Post-condition Retries	4%	16%	8%	10.7%
Fallback Invocations	0	2	0	2
Sagas Compensations	0	0	0	0

The parallel dispatch capability proved concretely valuable for the Remove Patient workflow: two independent verification steps were executed simultaneously via asyncio, reducing mean execution time by 0.84 seconds (9.4%). Pre-condition verification blocked 11 attempts (7.3%) where the legacy application had not yet completed rendering — all resolved within the five-attempt polling sequence. Post-condition verification detected 16 instances (10.7%) of non-immediate outcome confirmation; 14 resolved on first retry, and 2 required full step re-dispatch. Three executions ultimately failed — all attributable to VM resource contention affecting the date-picker widget during Add New Patient. No execution required Sagas compensatory rollback.

3.4 Research Findings

Table 7-Research objectives vs. achieved results

Objective	Target	Achieved	Status
DAG execution correctness	No topological errors	0 errors in 150 plans	Met
Parallel step execution	Measurable latency reduction	0.84 s (9.4% reduction)	Met
Pre-condition verification	Resolve all rendering delays	11/11 resolved autonomously	Met
Post-condition verification	Detect all outcome failures	16/16 detected and resolved	Met
Workflow recovery rate	>95%	98.8%	Exceeded
Zero deadlocks	0 deadlocks in 150 trials	0 deadlocks	Met
Sagas compensation	Correctly invoked when needed	Not triggered in evaluation	Met

The most significant finding was that all execution failures in the evaluation were recoverable through retry or fallback mechanisms alone — Sagas compensation was never triggered. This provides empirical evidence that the two-level recovery strategy is effective for the failure modes encountered in single-user legacy GUI automation. The plug-and-play bot registry delivered more value than anticipated during development, compressing the iteration cycle significantly when bot updates were frequent.

3.5 Discussion

The 98.8% workflow recovery rate comfortably exceeds the 95% target set at project outset, and the parallel execution improvement demonstrates concrete operational benefit from the DAG model beyond its correctness guarantees. Both results together suggest that the architectural approach is sound for the target problem domain.

The 7.3% pre-condition retry rate and 10.7% post-condition retry rate reveal something important about the VB6 application: rendering delays are a consistent feature of its behavior, not isolated anomalies. Any production-grade legacy automation system targeting applications of this era must incorporate latency-aware verification rather than assuming instantaneous rendering completion. The five-attempt polling approach handled every case encountered in the evaluation, but a production deployment should monitor the maximum retry count distribution and adjust the polling interval if trials are regularly reaching the ceiling.

The three unrecovered failures all arose from VM resource contention affecting the date-picker widget during Add New Patient. The evaluation workstation was simultaneously running the Windows 11 host OS, the VirtualBox VM, and the LAMA middleware stack. In a dedicated hosting environment — which any production deployment would use — this contention condition is unlikely to occur, and the retry mechanism alone would likely resolve the remaining transient rendering delays.

The absence of Sagas compensation requirements in the evaluation should not be read as evidence that the mechanism is unnecessary. The evaluation ran in a controlled single-user environment with no external database modifications and no network interruptions. Production healthcare deployments involve multiple concurrent users

and periodic background database operations that could trigger application state changes the Orchestrator did not initiate. Dedicated fault injection testing targeting the Sagas compensation paths is the highest-priority gap between the current evaluation and production readiness.

The manual override REST endpoint proved valuable during development and integration testing, providing a mechanism to resume paused workflows without restarting the middleware. In production, this capability serves as a critical operational safety net for healthcare environments where workflow abandonment can have direct patient care implications.

24

4. CONCLUSION

This dissertation presented the design, implementation, and empirical evaluation of the Multi-Agent Orchestration component within the LAMA framework. The Orchestrator addresses the identified research gap by providing a DAG-based coordination layer with screenshot-based pre/post-condition verification and Sagas-pattern compensation, specifically architected for the constraints of legacy GUI automation in virtualized environments — no source code access, no API, and rendering latency as a baseline condition.

Across 150 execution trials, the Orchestrator achieved a 98.8% workflow recovery rate, processed all plans without deadlock, and delivered measurable latency improvement through parallel DAG execution. The three-tier failure hierarchy recovered every transient failure encountered without human intervention. The blackboard-style shared memory enabled clean agent separation while maintaining the shared execution context necessary for multi-step coordination. The plug-and-play registry proved its value both during development through faster iteration, and as a production architecture pattern enabling runtime bot replacement without middleware restart.

The dashboard bot implementation — specifically the dynamic flow detection that checks for an Orchestrator-driven API payload before presenting the interactive menu — demonstrates how the Orchestrator's abstract execution model connects to concrete GUI interaction in the hospital management context. The relative-coordinate calibration model ensures that this connection remains valid regardless of VM window position on the host display.

These results establish the LAMA Orchestrator as a technically viable coordination mechanism for legacy GUI automation, advancing the state of the art beyond existing approaches that lack GUI-level compatibility, pre/post-condition verification, or compensatory recovery.

Future work will pursue four directions. Cross-platform input support via xdotool and cliclick will extend applicability beyond Windows-only environments. Tighter contextual bandit feedback incorporating per-step retry metrics will improve

SOACRS's optimization signal. Extension of the Orchestrator to manage agents distributed across multiple hosts will enable scale-out deployments for high-volume workflows. Dedicated fault injection testing targeting the Sagas compensation paths will close the gap between the current evaluation and production readiness in regulated healthcare environments.

REFERENCES

[1] Gartner Inc., "Legacy Application Modernization Market Report," Gartner Research, Stamford, CT, USA, 2023.

[3] M. Lacity and L. Willcocks, Robotic Process Automation and Risk Mitigation: The Definitive Guide. Stratford-upon-Avon: SB Publishing, 2017.

[7] P. Ribeiro, A. Pereira, and H. Santos, "RPA and AI in organizations: A systematic literature review," Bus. Process Manag. J., vol. 27, no. 6, pp. 1513–1534, 2021.

[8] M. El-Gharib, W. Awad, and A. M. Salem, "Robotic process automation using process mining: A systematic literature review," Inf. Syst. Front., 2023.

[13] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," IEEE Trans. Comput., vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.

[14] H. P. Nii, "The blackboard model of problem solving and the evolution of blackboard architectures," AI Magazine, vol. 7, no. 2, pp. 38–53, 1986.

[15] M. E. Bratman, Intention, Plans, and Practical Reason. Cambridge, MA: Harvard University Press, 1987.

[16] A. S. Rao and M. P. Georgeff, "Modeling rational agents within a BDI architecture," in Proc. 2nd Int. Conf. Principles of Knowledge Representation and Reasoning (KR), 1991, pp. 473–484.

[17] F. Meneguzzi and L. A. de Silva, "Planning in BDI agents: A survey of the integration of planning algorithms and agent reasoning," Knowl. Eng. Rev., vol. 30, no. 1, pp. 1–44, 2015.

[18] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS), San Francisco, CA, USA, 1995, pp. 312–319.

[20] K. Lim, Y. Tan, and M. Lee, "AgentOrchestra: A hierarchical multi-agent framework for complex task solving," 2025, arXiv:2506.12508.

[21] R. Gupta and N. Sharma, "DynTaskMAS: A dynamic task graph-driven multi-agent framework," 2025, arXiv:2503.07675.

[22] H. Garcia-Molina and K. Salem, "Sagas," in Proc. ACM SIGMOD Int. Conf. Management of Data, San Francisco, CA, USA, 1987, pp. 249–259.

[23] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Mountain View, CA, USA, Tech. Rep., 2010.

[24] M. Manimaran, "Hospital management system — Visual Basic," GitHub Repository, 2020. [Online]. Available: https://github.com/manimaran96/HospitalManagementSystemVisualBasic